CNCF Project Focus

Episode #8

Fluid

Elastic DATA abstraction

and

acceleration for BigData/AI applications in cloud

Christian Dussol

Read more

Cloud Native AI Infrastructure

Three questions behind running AI workloads on Kubernetes

1 **How do workloads share a GPU ?**
Previous episode - HAMi

2 **How do workloads get their data ?**
This episode - Fluid

3 **When is a workload admitted ?**
Closing episode

One read, one minute

A Job copies 443.5 MiB of files from a remote Apache mirror

First read: every byte travels from remote storage

FIRST READ

1 min 2.74 s

```
+ time cp -r /data/hbase ./
real  1m 2.74s
```

Source: official Fluid v1.0 docs · POSIX tutorial

Without a shared cache, every pod pays it again

Every new pod, replica and epoch re-reads it from remote storage.

IN PRODUCTION · CHINA MERCHANTS BANK

“when data is not ready, allocated accelerators wait.”

After GPU sharing (Episode 7), GPUs waiting on data are the next waste.

Source: CNCF case study, China Merchants Bank

Who needs a cache for their data?

Training datasets — GOOD FIT

Re-read every epoch.

Model weights — GOOD FIT

Loaded by every new replica.

Checkpoints — GOOD FIT

Read back to evaluate and deploy.

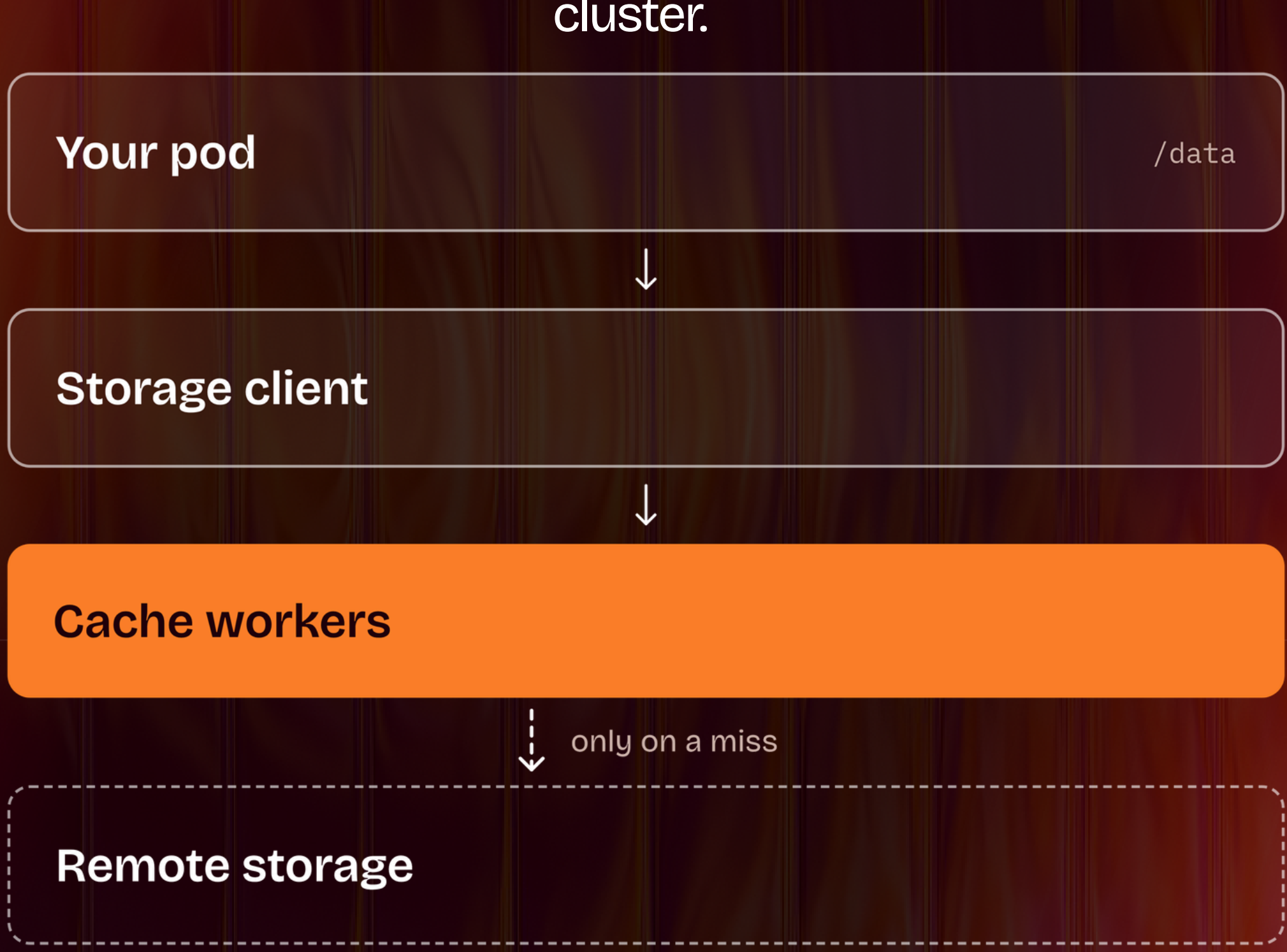
Data read once — IT DEPENDS

Cold unless preloaded.

A cache pays off when data is read again

How a read is served

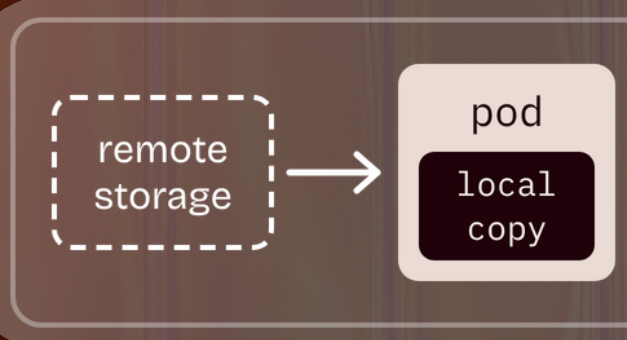
With a Fluid cache runtime, every read goes through a cache in the cluster.



Four ways to feed a pod

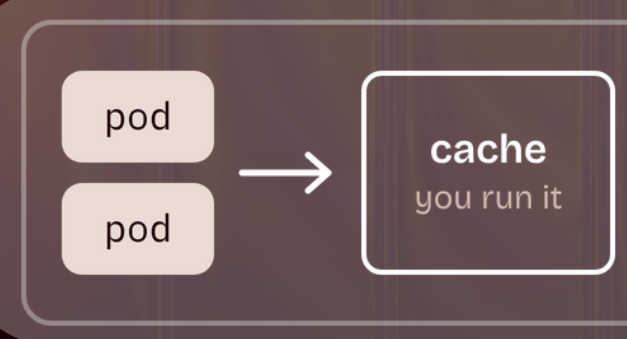
Direct PVC

Every read is remote



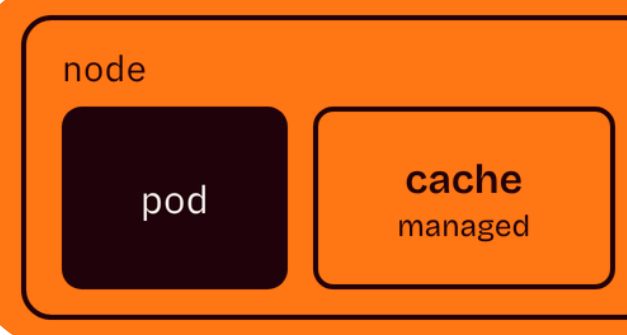
Init container copy

Each pod copies the data first



Hand-run cache

Shared cache, you operate it



Fluid

A cache engine, orchestrated by Kubernetes

What is Fluid ?

“Fluid is an open source Kubernetes-native
Distributed Dataset Orchestrator
and
Accelerator for data-intensive applications, such as big data and AI applications.”

July 2020
First commit

28 April 2021
Accepted into the CNCF Sandbox

8 January 2026
Entered CNCF Incubating

How Fluid speeds up data

Fluid does not replace your storage.
It turns a cache engine (Alluxio, JuiceFS…) into Kubernetes objects:
declare it, fill it, keep it, place pods next to it.

DECLARE — **Say where the data is**

CACHE — **Keep a copy close**

PRELOAD — **Fill it before the pod starts**

PLACE — **Put the pod next to the cache**

Declare the data

DECLARE → CACHE → PRELOAD → PLACE

Dataset
where the data is

Runtime
who serves it

↓ Fluid binds them: `NotBound` → `Bound`

PV + PVC
created by Fluid

→

Your pod
mounts it at `/data`

```
$ kubectl get dataset hbase
```

ONCE BOUND

```
NAME    UFS TOTAL SIZE   CACHE CAPACITY   PHASE
hbase   443.5MiB         4GiB             Bound
```

Source: Fluid v1.0 docs · POSIX tutorial (excerpt)

Pay the remote read once

DECLARE → CACHE → PRELOAD → PLACE

Run 2 reads from the cache, not from the remote mirror

RUN 1 · FIRST READ

Job pod

↓

Cache · empty · miss

↓

Remote mirror

1 min 2.74 s

RUN 2 · SECOND READ

Job pod

↓

Cache · full · hit

Remote mirror

2.94 s

```
+ time cp -r /data/hbase ./
real   0m 2.94s
```

Source: Fluid v1.0 docs · POSIX tutorial

Warm the cache before the pod starts

DECLARE → CACHE → PRELOAD → PLACE

WITHOUT DATALOAD

Pod starts
cache empty

→

Cold read
from remote storage

WITH DATALOAD

DataLoad
0% → 100%
of 1.92GiB cached

→

Pod starts
cache full

→

Warm read
from the cache

ALL IT TAKES (EXCERPT)

```
kind: DataLoad
metadata:
  name: spark-dataload
spec:
  dataset:
    name: spark
    namespace: default
```

Source: Fluid v1.0 docs · Data Preloading (excerpt)

Put the pod next to the cache

DECLARE → CACHE → PRELOAD → PLACE

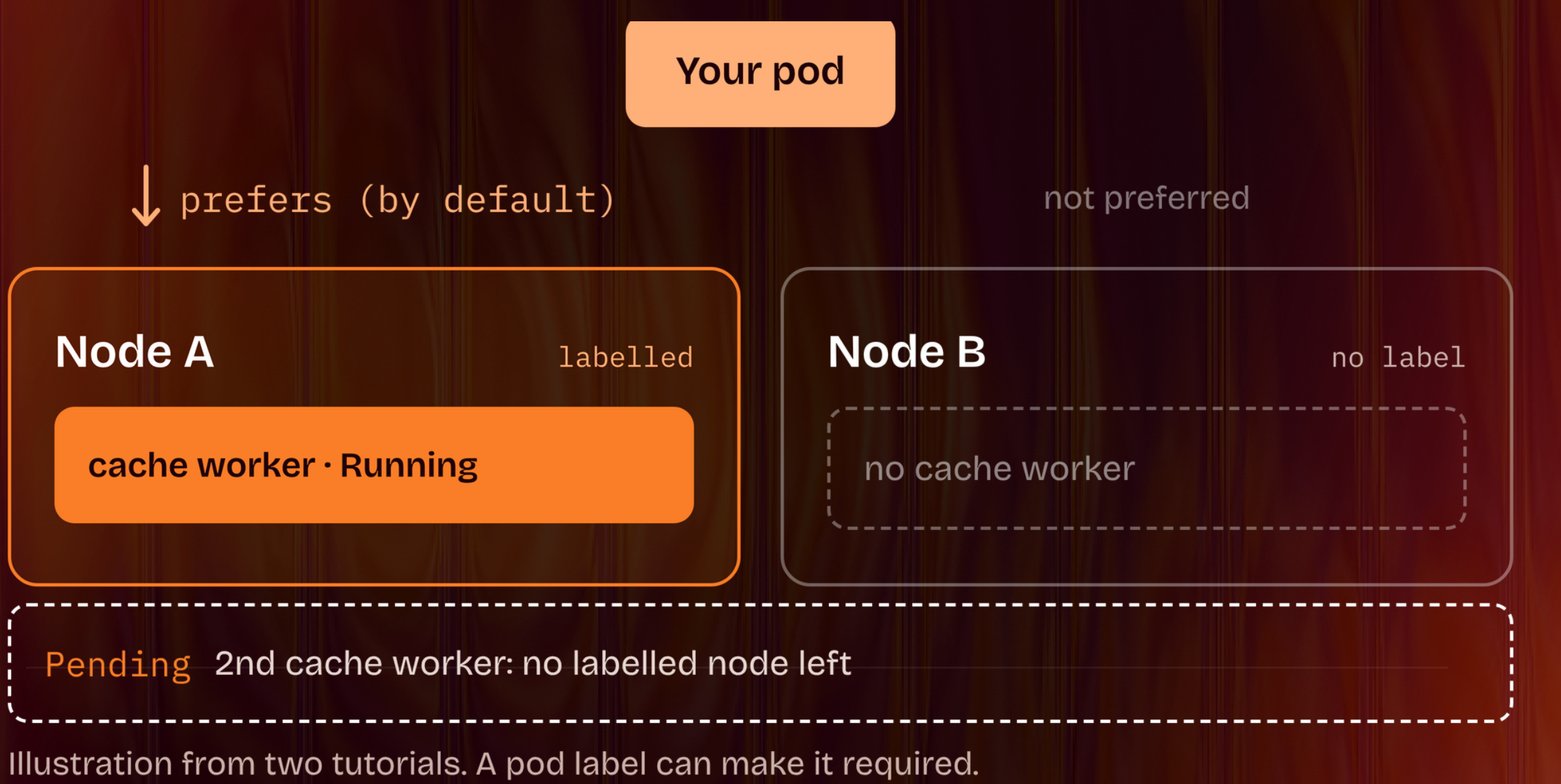


Illustration from two tutorials. A pod label can make it required.

POD PLACEMENT · AFFINITY FLUID ADDS TO THE POD

```
preferredDuringSchedulingIgnoredDuringExecution:
- preference:
    matchExpressions:
    - key: fluid.io/s-default-hbase
      operator: In
      values:
      - "true"
  weight: 100
```

What Fluid does not do

01 **It is not a storage system**

Nor a cache engine itself

02 **The first read stays cold**

Unless the data is preloaded

03 **The cache is bounded**

4GiB in the tutorials

04 **Acceleration depends on the runtime**

Not every runtime caches.

Official Fluid tutorials

START

Install Fluid on a cluster

Quick Start

CACHE

Read the same data twice

Accelerate Data Accessing (POSIX)

PRELOAD

Warm the cache before the pod starts

Data Preloading

PLACE

Run the pod next to the cache

Cache Co-locality

fluid-cloudnative.github.io/docs

Links in the comments